Normalization in RDBMS

Dr Banani Das

Introduction Normalization (contd.)

Normalization is a process specifically designed for relational databases, proposed by E.F. Codd.

Normalization is a systematic approach to organize data within a database to:

- To reduce redundancy
- To eliminate anomalies
- To ensure data integrity

The process involves breaking down large tables into smaller, well-structured ones and defining relationships between them.

Introduction Normalization (contd.)

Importance of Normalizationn

- **Reduces Data Redundancy** by storing duplicate data efficiently
- **Improves Data Integrity** by ensuring the accuracy and consistency
- **Simplifies Database Design** by following a clear structure
- **Optimizes Performance** by reducing the chance of anomalies

Normalization aims to reduce redundancy and minimize insertion, update, and deletion anomalies.

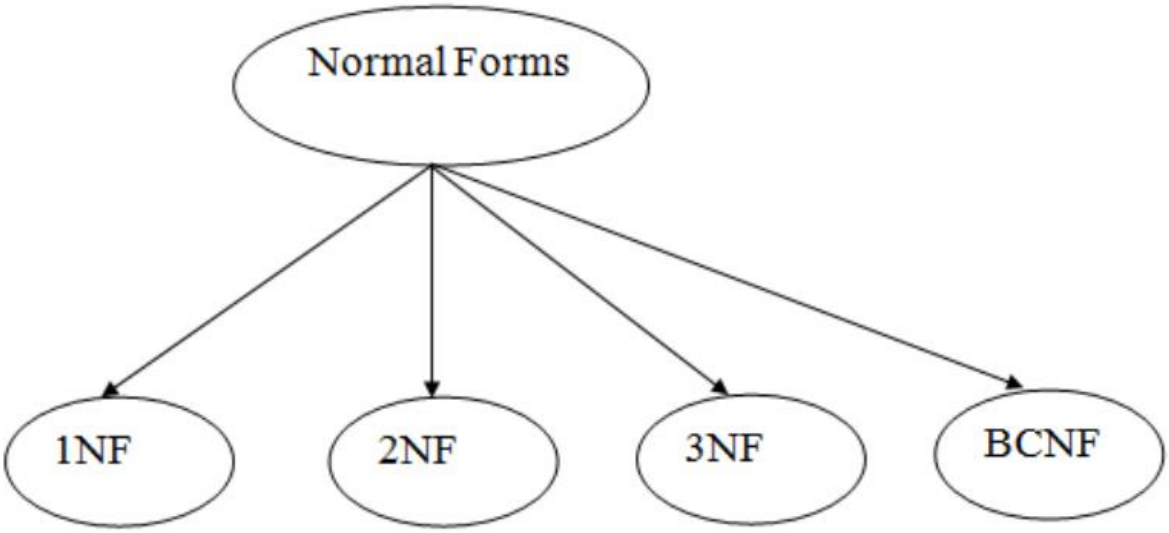
Anomalies are the inconsistencies or errors that arise due to improper database design, often involving data redundancy.

- Insertion Anomaly: Difficulty adding data due to missing other data.
- Deletion Anomaly: Unintended loss of data due to deletion.
- Update Anomaly: Data inconsistency due to partial update.

Normal Forms

Normalization is done using **Normal Forms.**

Normal Forms are different stages of normalization, and each stage imposes certain rules to improve the structure and performance of a database.



Normal Forms

Normal Form	Description
1NF	A relation is in 1NF if it contains an atomic value.
2NF	A relation will be in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key.
3NF	A relation will be in 3NF if it is in 2NF and no Transitive dependency exists.
BCNF	An advance and stricter version of 3NF

First Normal Form (1NF)

- It disallow multivalued attributes, composite attributes and their combinations.
- It stated that the domain of an attribute must be a single value.
- It disallows relations within relations.
- It allows only a single atomic (or indivisible) values

First Normal Form (1NF)

Let's consider the following example:

DEPARTMENT

D_Name	D_Number	MMGR_SSN	DLocations
Research	5	3334	{Kolkata, Mumbai, Assam}
Headquarters	1	6678	{New Delhi}

FD: D_Number → D_Name, DMGR_SSN, D_Locations
and D_Number is the Primary Key

From relation state it is clear that the relation (DEPARTMENT) is **not in 1NF** because **Dlocation** is **not atomic attribute**. (one cell must contains only one value)

First Normal Form (1NF) – contd.

There are 3 techniques to convert DEPARTMENT relation into 1NF:

Method 1: Decomposes the non-1NF relation into two 1NF relations

DEPARTMENT

D_Name	D_Number	MMGR_SSN	DLocations
Research	5	3334	{Kolkata, Mumbai, Assam}
Headquarters	1	6678	{New Delhi}

DEPARTMENT

D_Name	D_Number	MMGR_SSN
Research	5	3334
Headquarters	1	6678

Dept_Locations

D_Number	DLocations
5	{Kolkata}
5	{Mumbai}
5	{Assam}
1	{New Delhi}

First Normal Form (1NF) – contd.

Method 2: Expand the Key

DEPARTMENT

D_Name	D_Number	MMGR_SSN	DLocations
Research	5	3334	{Kolkata, Mumbai, Assam}
Headquarters	1	6678	{New Delhi}

↓

DEPARTMENT

D_Name	D_Number	MMGR_SSN	DLocations
Research	5	3334	{Kolkata}
Research	5	3334	{Mumbai}
Research	5	3334	{Assam}
Headquarters	1	6678	{New Delhi}

First Normal Form (1NF) – contd.

Method 3: If the maximum number of values is known for the attribute, i.e. if at most 3 locations can exist in the example, then add 3 columns for locations.

DEPARTMENT

D_Name	D_Number	MMGR_SSN	DLocations
Research	5	3334	{Kolkata, Mumbai, Assam}
Headquarters	1	6678	{New Delhi}

↓

DEPARTMENT

D_Name	D_Number	MMGR_SSN	Dlocation_1	Dlocation_2	Dlocations_3
Research	5	3334	Kolkata	Mumbai	Assam
Headquarters	1	6678	New Delhi	NULL	NULL

Second Normal Form (2NF)

- It is based on the concept of full functional dependency
- A functional dependency X →Y is a full functional dependent if removal of any attribute A from X means that the dependency does not hold any more.
- A relation, R is in 2NF if every nonprime attribute A in R is fully full functional dependent on the primary key of R.

  i.e. all non-key attributes are fully functional dependent on the primary key

Second Normal Form (2NF) – contd.

Let's consider the following example:

Employee_Project

SSN	Proj_Number	Hours	Emp_Name	Proj_Name	Proj_Location

FD1: {SSN, Proj_Number) → Hours
FD2: SSN → Emp_Name
FD3: Proj_Number → {Proj_Name, Proj_Location}

The Employe_Project relation is in 1NF but not in 2NF because:

- In FD2, the non-prime attribute Emp_Name violates the fully functional dependency. As SSN → Emp_Name is partial.
- In FD3 also violates 2NF as, {Proj_Name, Proj_Location} is partially dependent on the primary key (i.e. only Proj_Number)

Second Normal Form (2NF) – contd.

Employee_Project relation can be decomposed as follows:

Employee_Project

SSN	Proj_Number	Hours	Emp_Name	Proj_Name	Proj_Location

EP_1

SSN	Proj_Number	Hours

EP_2

SSN	Emp_Name

EP_3

Proj_Number	Proj_Name	Proj_Location

Third Normal Form (3NF)

- It is based on the concept of transitive dependency
- A functional dependency X →Y is a relational schema R is a transitive dependency if there is a set of attributes Z, that is neither a candidate key nor a subset of any key of R and both X → Z and Z → Y holds.
- A relation, R is in 3NF if it satisfies 2NF and no non-prime attribute of R is transitively dependent on the primary key.

Third Normal Form (3NF) – contd.

Let's consider the following example:

Employee_Department

E_Name	SSN	Birth_Date	Address	Dept_Number	Dept_Name	DMGRSSN

ED_1

E_Name	SSN	Birth_Date	Address	Dept_Number

ED_2

Dept_Number	Dept_Name	DMGRSSN

Boyce-Codd Normal Form (BCNF)

- It is the advance and stricter version of 3NF.
- A relation is in BCNF if every functional dependency X → Y, X is the super key of the table
- For BCNF, the relation should be in 3NF and in every FDs, L.H.S should be super key.
- For any A → B, A should be super key

i.e. A cannot be non-prime attribute, if B is a prime attribute

Boyce-Codd Normal Form (BCNF)-contd.

Let's consider the following example:

Emp_Dept_details

Emp_ID	Semp_Country	Emp_Dept	Dept_Type	Emp_Dept_No

FD1: EMP_ID → EMP_COUNTRY
FD2: EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither **EMP_DEPT nor EMP_ID alone are keys.**

Emp_Country

Emp_ID	Semp_Country

Emp_Dept

Emp_Dept	Dept_Type	Emp_Dept_No

Emp_Dept_Mapping

Emp_ID	Emp_Dept

Thank You